

DevOps and Cloud

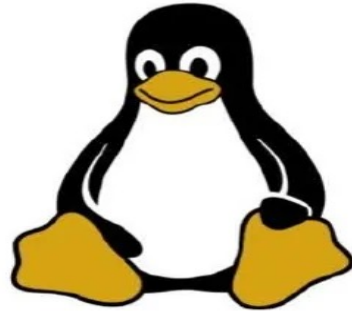
Bootcamp Session – Git, Linux

<https://topmate.io/cloud>

<https://www.youtube.com/@DevOps-Cloud>

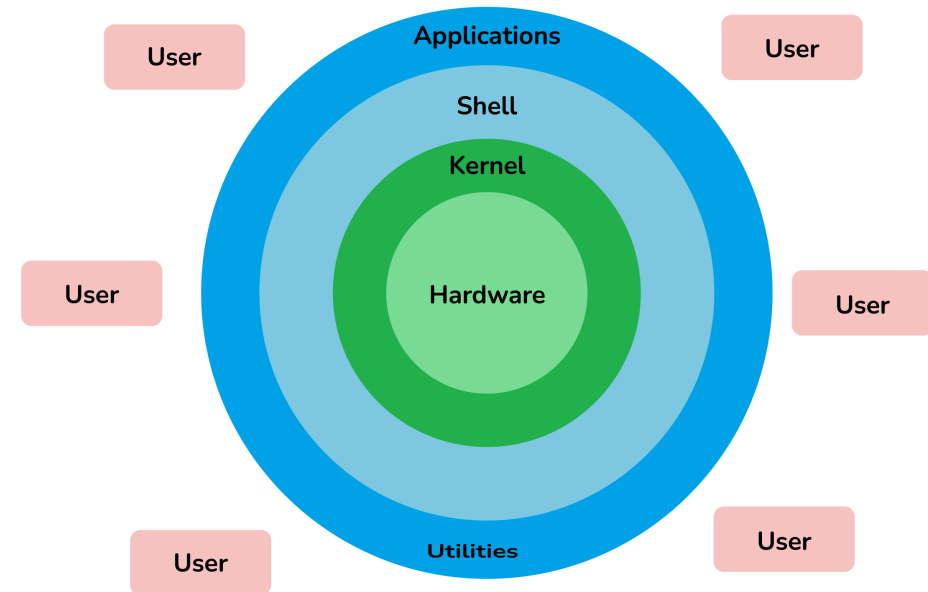
Linux

What Is Linux?

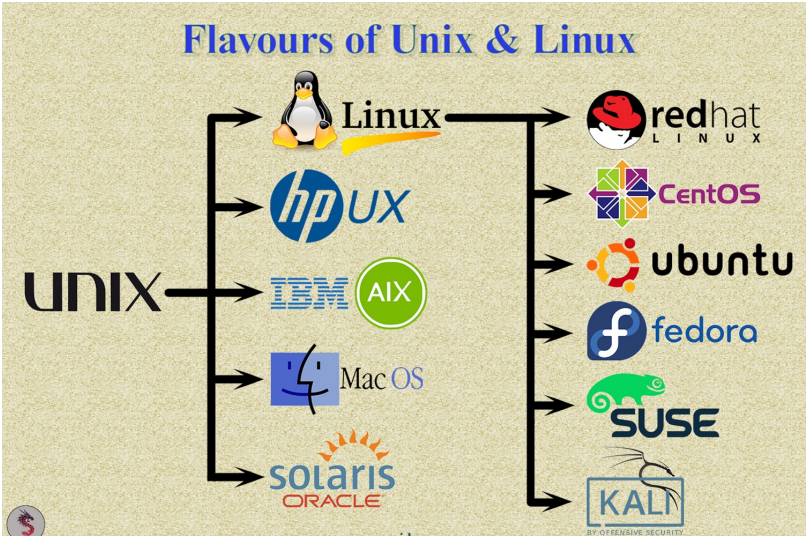


Linux is an operating system's kernel. You might have heard of UNIX. Well, Linux is a UNIX clone. But it was actually created by Linus Torvalds from Scratch. Linux is free and open-source, that means that you can simply change anything in Linux and redistribute it in your own name! There are several Linux Distributions, commonly called “distros”.

- Ubuntu Linux
- Red Hat Enterprise Linux
- Linux Mint
- Debian
- Fedora



Unix flavours



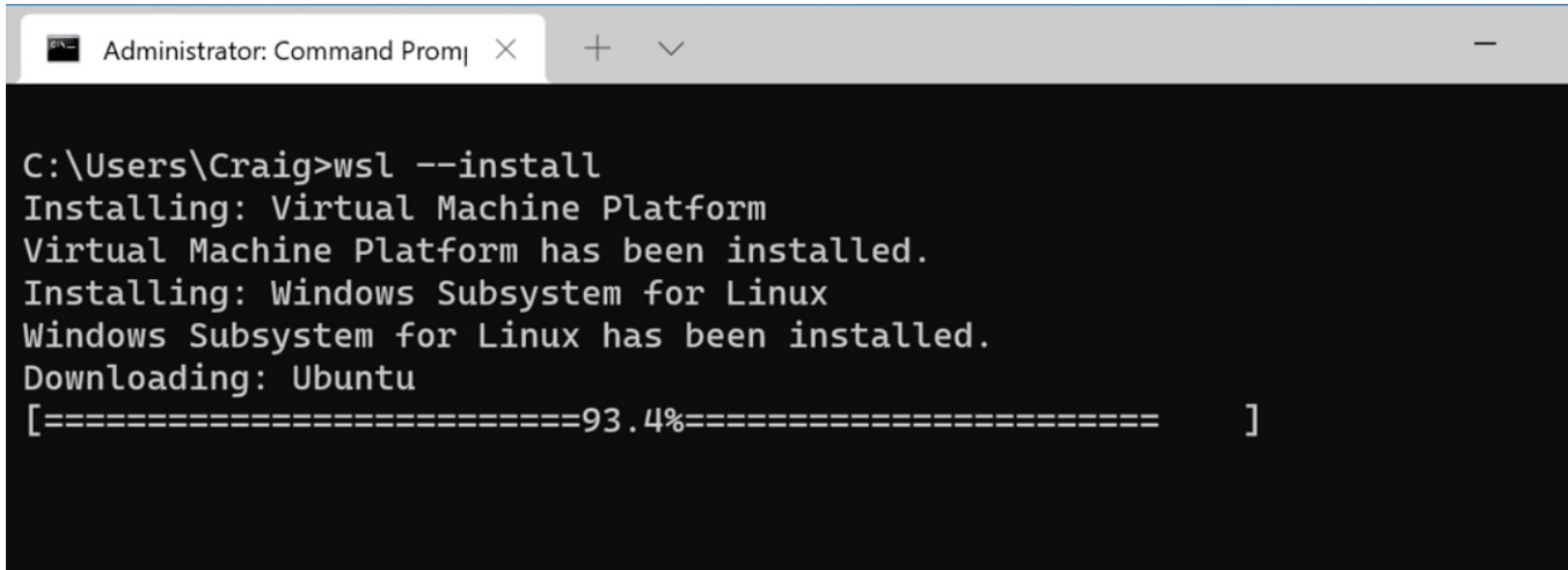
Features	CentOS	Ubuntu
		
Package Command	RPM and YUM	APT-GET
Variant	Based on Red Hat Enterprise Linux (RHEL)	Based on Debian
Percentages of websites using (as per this site)	20.4%	34.8%
Release Cycle	Long period	Short period
Security	Secure out of the box	Less secure (out of the box) compare to CentOS
Download Link	Download CentOS	Download Ubuntu

RHEL/centOS – used in Large Corporations

Ubuntu – used in Small Businesses and Startups

WSL (Windows Subsystem for Linux)

<https://learn.microsoft.com/en-us/windows/wsl/install>

A screenshot of a Windows Command Prompt window titled "Administrator: Command Prompt". The window has a dark background and white text. The command prompt shows the execution of the command "wsl --install". The output indicates the installation of the Virtual Machine Platform and the Windows Subsystem for Linux, followed by the download of Ubuntu. A progress bar for the Ubuntu download is shown at 93.4% completion.

```
C:\Users\Craig>wsl --install
Installing: Virtual Machine Platform
Virtual Machine Platform has been installed.
Installing: Windows Subsystem for Linux
Windows Subsystem for Linux has been installed.
Downloading: Ubuntu
[=====93.4%=====]
```

Basic Linux Commands

Command	Description
ls	Lists all files and directories in the present working directory
ls - R	Lists files in sub-directories as well
ls - a	Lists hidden files as well
ls - al	Lists files and directories with detailed information like permissions, size, owner, etc.
cat > filename	Creates a new file
cat filename	Displays the file content
cat file1 file2 > file3	Joins two files (file1, file2) and stores the output in a new file (file3)
mv file "new file path"	Moves the files to the new location
mv filename new_file_name	Renames the file to a new filename
sudo	Allows regular users to run programs with the security privileges of the superuser or root
rm filename	Deletes a file
man	Gives help information on a command
history	Gives a list of all past basic Linux commands list typed in the current terminal session
clear	Clears the terminal
mkdir directoryname	Creates a new directory in the present working directory or a at the specified path
rmdir	Deletes a directory
mv	Renames a directory

Linux Top 50 Scenario & Commands

- 1.**ls** - The most frequently used command in Linux to list directories
- 2.**pwd** - Print working directory command in Linux
- 3.**cd** - Linux command to navigate through directories
- 4.**mkdir** - Command used to create directories in Linux
- 5.**mv** - Move or rename files in Linux
- 6.**cp** - Similar usage as mv but for copying files in Linux
- 7.**rm** - Delete files or directories
- 8.**touch** - Create blank/empty files
- 9.**ln** - Create symbolic links (shortcuts) to other files
- 10.**cat** - Display file contents on the terminal
- 11.**clear** - Clear the terminal display
- 12.**echo** - Print any text that follows the command
- 13.**less** - Linux command to display paged outputs in the terminal
- 14.**man** - Access manual pages for all Linux commands
- 15.**uname** - Linux command to get basic information about the OS
- 16.**whoami** - Get the active username
- 17.**tar** - Command to extract and compress files in Linux
- 18.**grep** - Search for a string within an output
- 19.**head** - Return the specified number of lines from the top
- 20.**tail** - Return the specified number of lines from the bottom
- 21.**diff** - Find the difference between two files
- 22.**cmp** - Allows you to check if two files are identical
- 23.**comm** - Combines the functionality of diff and cmp
- 24.**sort** - Linux command to sort the content of a file while outputting
- 25.**export** - Export environment variables in Linux

- 1.**zip** - Zip files in Linux
- 2.**unzip** - Unzip files in Linux
- 3.**ssh** - Secure Shell command in Linux
- 4.**service** - Linux command to start and stop services
- 5.**ps** - Display active processes
- 6.**kill and killall** - Kill active processes by process ID or name
- 7.**df** - Display disk filesystem information
- 8.**mount** - Mount file systems in Linux
- 9.**chmod** - Command to change file permissions
- 10.**chown** - Command for granting ownership of files or folders
- 11.**ifconfig** - Display network interfaces and IP addresses
- 12.**traceroute** - Trace all the network hops to reach the destination
- 13.**wget** - Direct download files from the internet
- 14.**ufw** - Firewall command
- 15.**iptables** - Base firewall for all other firewall utilities to interface with
- 16.**apt, pacman, yum, rpm** - Package managers depending on the distro
- 17.**sudo** - Command to escalate privileges in Linux
- 18.**cal** - View a command-line calendar
- 19.**alias** - Create custom shortcuts for your regularly used commands
- 20.**dd** - Majorly used for creating bootable USB sticks
- 21.**whereis** - Locate the binary, source, and manual pages for a command
- 22.**whatis** - Find what a command is used for
- 23.**top** - View active processes live with their system usage
- 24.**useradd and usermod** - Add new user or change existing users data
- 25.**passwd** - Create or update passwords for existing users

Reference - <https://www.digitalocean.com/community/tutorials/linux-commands>

File Attributes

```
zaira@Zaira:~/freeCodeCamp$ ls -l
total 3856
-rw-r--r--      1 zaira zaira      89 Apr  5 20:46 CODE_OF_CONDUCT.md
-rw-r--r--      1 zaira zaira     210 Apr  5 20:46 CONTRIBUTING.md
-rw-r--r--      1 zaira zaira    1513 Apr  5 20:46 LICENSE.md
-rw-r--r--      1 zaira zaira   19933 Apr  5 20:46 README.md
drwxr-xr-x      4 zaira zaira    4096 Apr  6 22:45 api-server
-rw-r--r--      1 zaira zaira      67 Apr  5 20:46 babel.config.js
drwxr-xr-x     10 zaira zaira    4096 Apr  6 22:55 client
drwxr-xr-x      5 zaira zaira    4096 Apr  6 22:54 config
```

┌───┐
│ │
└───┘
MODE

┌───┐
│ │
└───┘

┌───┐
│ │
└───┘

OWNER

┌───┐
│ │
└───┘

GROUP

┌───┐
│ │
└───┘

SIZE

┌───┐
│ │
└───┘

┌───┐
│ │
└───┘

MODIFICATION DATE

┌───┐
│ │
└───┘

┌───┐
│ │
└───┘

FILE/FOLDER NAME

7

https://topmate.io/cloud

https://www.youtube.com/@DevOps-Cloud

File Permissions

Octal Representation

0	000	-	-	-	No permissions
1	001	-	-	x	Only Execute
2	010	-	w	-	Only Write
3	011	-	w	x	Write and Execute
4	100	r	-	-	Only Read
5	101	r	-	x	Read and Execute
6	110	r	w	-	Read and Write
7	111	r	w	x	Read, Write and Execute

Shell Script

A shell script is a text file that contains a sequence of commands for a UNIX-based operating system. It is called a shell script because it combines a sequence of commands, that would otherwise have to be typed into the keyboard one at a time, into a single script.

The shell is the operating system's command-line interface (CLI) and interpreter for the set of commands that are used to communicate with the system.

Reference:

<https://www.freecodecamp.org/news/bash-scripting-tutorial-linux-shell-script-and-command-line-for-beginners/>

Shell Script Example

SCRIPT



first.sh

```
#!/bin/sh
# This is a comment!
echo Bootcamp Learners
```

second.sh

```
#!/bin/bash
salary=1000
expenses=800
#Check if salary and expenses are equal
if [ $salary == $expenses ];
then
echo "Salary and expenses are equal"
#Check if salary and expenses are not equal
elif [ $salary != $expenses ];
then
echo "Salary and expenses are not equal"
fi
```

EXECUTION

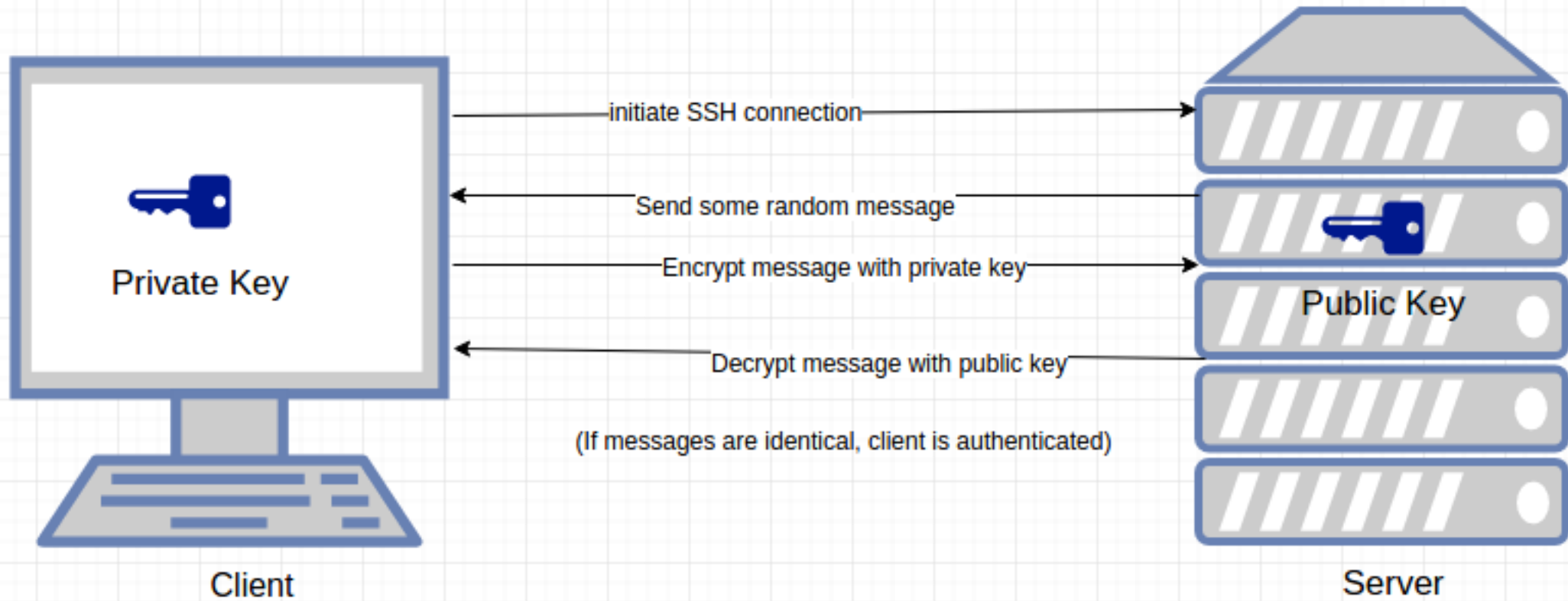


```
chmod 755 first.sh
./first.sh
```

```
chmod 755 second.sh
./second.sh
```

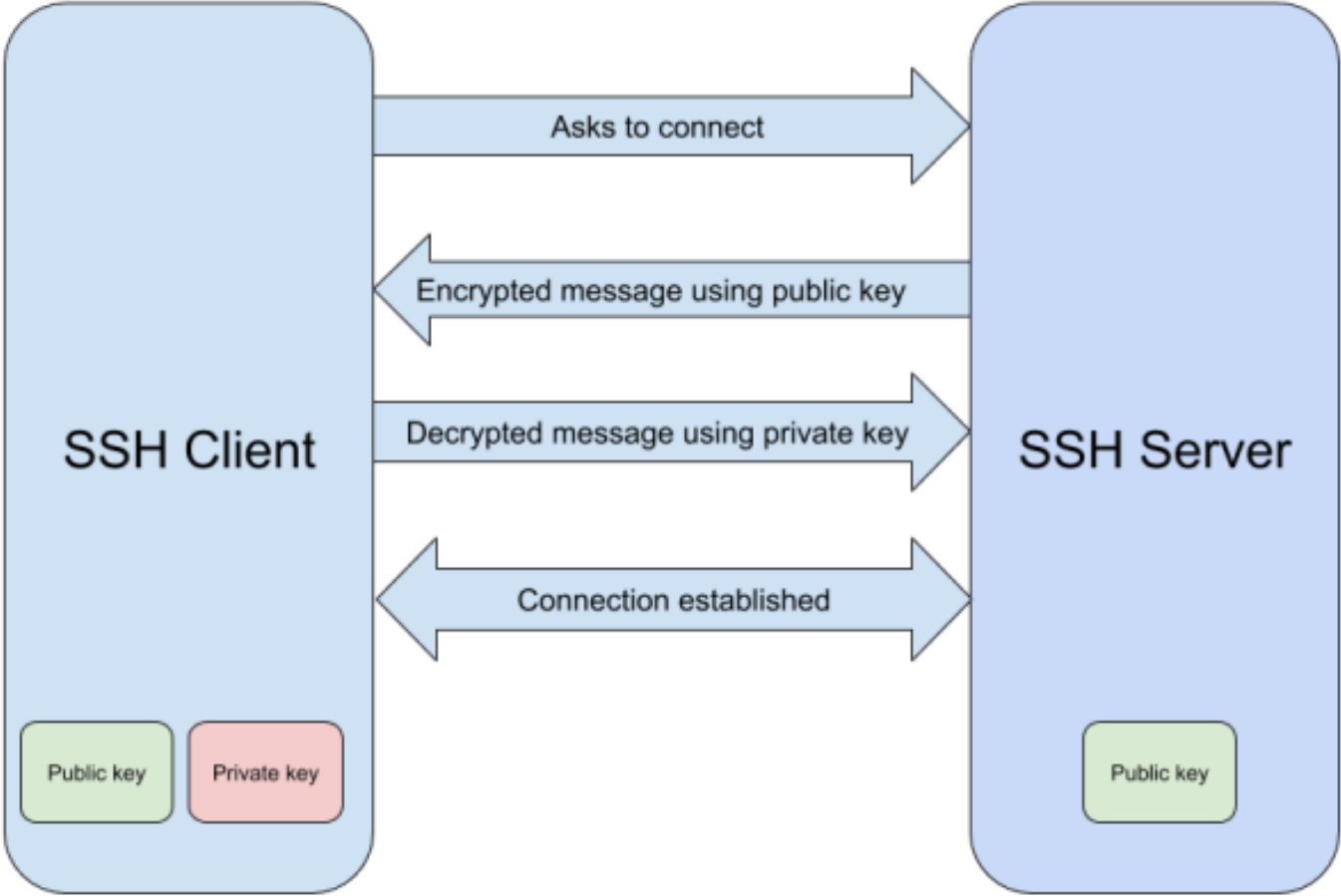
Public and Private Key

SSH Authentication



Refer <https://www.ssh.com/academy/ssh/keygen>

How ssh key works



Package Manager - YUM

Yum is a package manager used on Red Hat-based Linux distributions such as Red Hat Enterprise Linux (RHEL), CentOS, and Fedora. Below are some common Linux YUM commands:

1. Update Package Lists:

1. `yum update`: Updates all installed packages to their latest versions.

2. Search for Packages:

1. `yum search package-name`: Searches for packages matching the given name.

3. Install a Package:

1. `yum install package-name`: Installs the specified package and its dependencies.
2. Example: `yum install httpd` to install the Apache HTTP server.

4. Remove a Package:

1. `yum remove package-name`: Removes the specified package.
2. Example: `yum remove httpd` to remove the Apache HTTP server.

5. List Installed Packages:

1. `yum list installed`: Lists all installed packages.

6. List Available Packages:

1. `yum list available`: Lists all available packages.

7. List Repositories:

1. `yum repolist`: Lists enabled repositories.

Most Common Linux packages

The "most common" Linux packages can vary depending on the specific use case, distribution, and user preferences. However, there are several packages that are widely used across many Linux distributions due to their essential roles in system operation and software development. Here are some of the most common Linux packages:

- 1.GNU Core Utilities (coreutils):** This package includes essential command-line utilities like ls, cd, pwd, cp, mv, rm, and more.
- 2.Bash (Bourne Again Shell):** The default command-line shell on most Linux systems, used for scripting and interacting with the system.
- 3.OpenSSH:** Provides secure remote access and remote execution capabilities.
- 4.GNU Compiler Collection (gcc):** A suite of compilers for various programming languages, including C, C++, and Fortran.
- 5.Python:** A versatile and widely-used scripting and programming language.
- 6.Vim/Emacs:** Popular text editors used for coding and text file editing.
- 7.curl and wget:** Command-line tools for downloading files from the internet.
- 8.Git:** The distributed version control system widely used for software development.
- 9.Apache HTTP Server (httpd):** A popular web server software used to host websites.
- 10.Nginx:** Another widely used web server and reverse proxy server.
- 11.MySQL and PostgreSQL:** Popular relational database management systems.
- 12.Docker:** A containerization platform for developing, shipping, and running applications.
- 13.Node.js:** A JavaScript runtime used for building server-side applications.
- 14.Java Runtime Environment (JRE):** Required for running Java applications.
- 15.PHP:** A scripting language commonly used for web development.

Install Apache Web Server

```
yum install httpd
```

```
[root@ip-172-31-8-22 ~]# yum install httpd
Last metadata expiration check: 0:00:50 ago on Sat Sep 23 04:39:41 2023.
Dependencies resolved.
=====
Package                                                    Architecture
=====
Installing:
  httpd                                                    x86_64
Installing dependencies:
  apr                                                    x86_64
  apr-util                                                x86_64
  generic-logos-httpd                                    noarch
```

```
service httpd start
```

```
[root@ip-172-31-8-22 ~]# service httpd start
Redirecting to /bin/systemctl start httpd.service
[root@ip-172-31-8-22 ~]# █
```

```
cd /var/www/html
```

```
[root@ip-172-31-8-22 ~]# cd /var/www/html
[root@ip-172-31-8-22 html]# ls -latr
total 0
drwxr-xr-x. 2 root root  6 Mar 16  2023 .
drwxr-xr-x. 4 root root 33 Sep 23 04:40 ..
[root@ip-172-31-8-22 html]# █
```

Run Apache Web Server

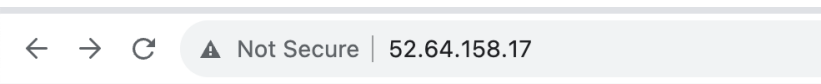
```
cd /var/www/html
```

Create a file index.html as below

```
<html>
<h1>
Hello Bootcamp Explorers! <br />
Welcome to the BOOTCAMP <br />
<a href="https://www.youtube.com/channel/UCYDlBuxE7BEtYR5r4q3WRrQ">Here's DevOps & Cloud Youtube</a>
</h1>
</html>
```

```
service httpd restart
```

```
[root@ip-172-31-8-22 html]# cat index.html
<html> <h1>
    Hello Bootcamp Explorers! <br />
    Welcome to the BOOTCAMP <br />
    <a href="https://www.youtube.com/channel/UCYDlBuxE7BEtYR5r4q3WRrQ">Here's DevOps & Cloud Youtube</a>
</h1> </html>
```



Hello Bootcamp Explorers!
Welcome to the BOOTCAMP
Here's DevOps & Cloud Youtube

‘vi’ editor

The vi text editor is a powerful and widely used text editor in the Linux and Unix world. It operates in different modes: command mode, insert mode, and visual mode. Here are some common vi commands you'll need to know:

1. Opening and Saving Files:

- ``vi filename``: Opens a file named "filename" in Vi.
- ``:w``: Save changes to the file.
- ``:w filename``: Save the file with a new name.
- ``:q``: Quit Vi.
- ``:q!``: Quit Vi without saving changes.
- ``:wq`` or ``:x``: Save changes and quit Vi.

2. Navigation (Command Mode):

- ``h``: Move the cursor left.
- ``j``: Move the cursor down.
- ``k``: Move the cursor up.
- ``l``: Move the cursor right.
- ``0`` (zero): Move to the beginning of the current line.
- ``$``: Move to the end of the current line.
- ``G``: Move to the end of the file.
- ``:n``: Move to line "n" (replace "n" with a line number).
- ``Ctrl + f``: Move forward one page.
- ``Ctrl + b``: Move backward one page.
- ``/search_term``: Search for "search_term" (press ``n`` to find the next occurrence).
- ``?search_term``: Search backward.

3. Editing (Command Mode):

- ``i``: Enter insert mode before the cursor.
- ``I``: Enter insert mode at the beginning of the current line.
- ``a``: Enter insert mode after the cursor.
- ``A``: Enter insert mode at the end of the current line.
- ``o``: Open a new line below the current line and enter insert mode.
- ``O``: Open a new line above the current line and enter insert mode.
- ``x``: Delete the character under the cursor.
- ``dd``: Delete the current line.
- ``yy``: Yank (copy) the current line.
- ``p``: Paste the last yanked text.

4. Exiting Insert Mode:

- ``Esc``: Exit insert mode and return to command mode.

5. Visual Mode:

- ``v``: Enter visual mode to select text.
- ``V``: Enter visual line mode to select whole lines.
- After selecting text, you can cut (``d``), copy (``y``), or paste (``p``) the selected text.

6. Undo and Redo:

- ``u``: Undo the last change.
- ``Ctrl + r``: Redo (opposite of undo).

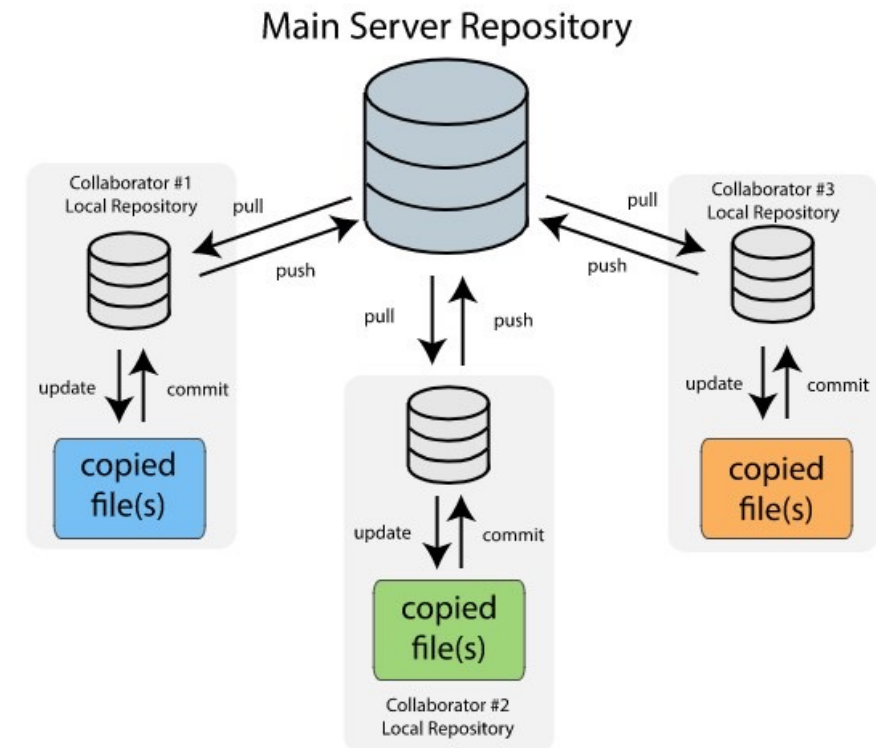
Version Control

Version Control

Version control, also known as source control, is the practice of tracking and managing changes to software code.

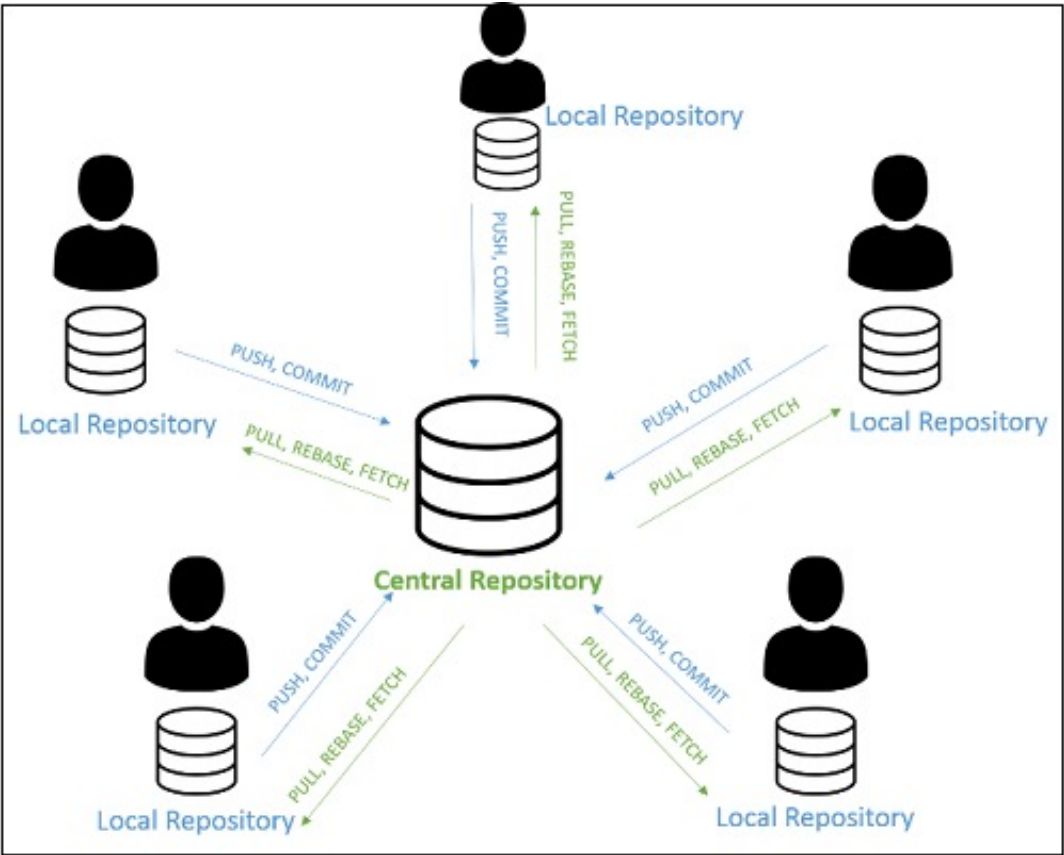
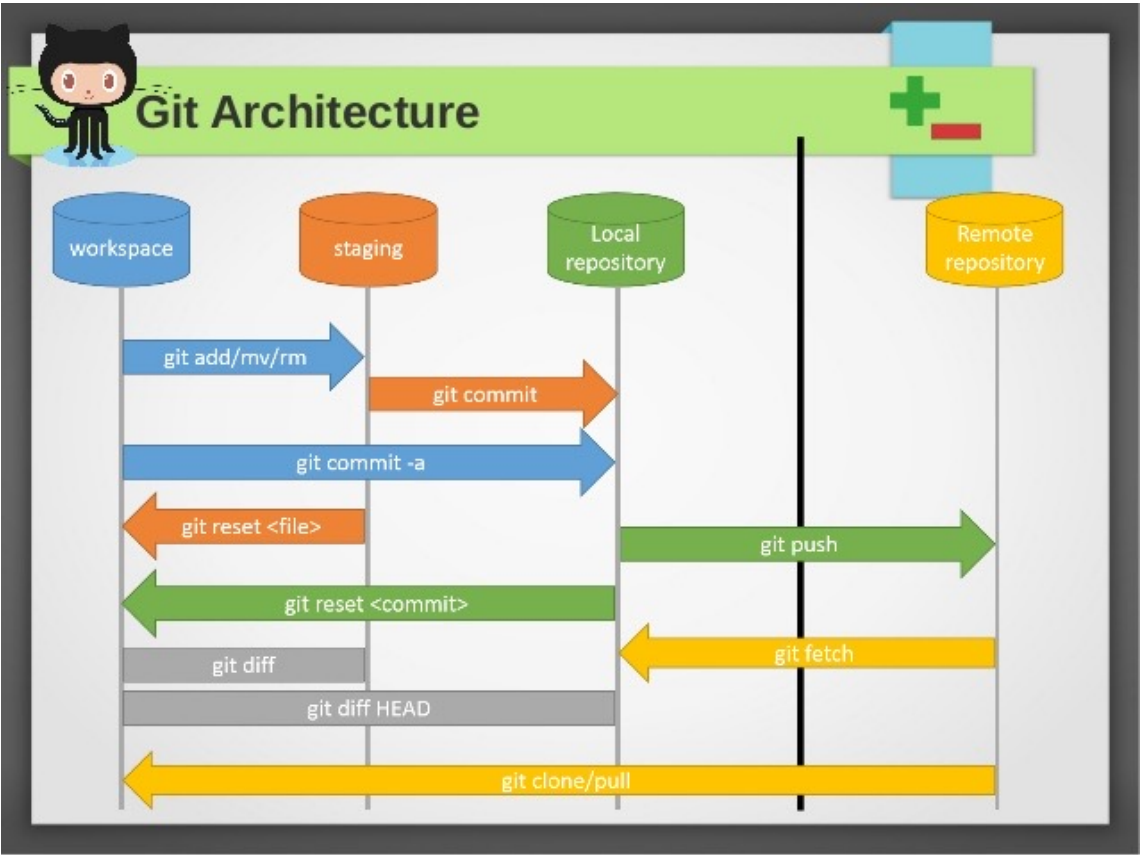
Version control systems are software tools that help software teams manage changes to source code over time. Version control software keeps track of every modification to the code in a special kind of database.

Distributed Version Control



A Distributed Version Control System. Each collaborator has a local copy of the repository, so no Internet connection is required.

Git



Different types of Branching Strategies

Common Branching Strategies

1. Centralized Workflow

- Simple:** A single main branch (usually master or main) is used for all development.
- Pros:** Easy to manage, suitable for small teams.
- Cons:** Can lead to merge conflicts and delays, especially for large projects.

2. Feature Branch Workflow

- Common:** A separate branch is created for each new feature or bug fix.
- Pros:** Isolates changes, reduces merge conflicts, and allows parallel development.
- Cons:** Can become complex for large projects with many branches.

3. Gitflow Workflow

- Structured:** Defines specific branches for development, staging, and production.
- Branches:**
 - master: Production-ready code
 - develop: Main development branch
 - feature: For new features
 - release: For preparing releases
 - hotfix: For urgent bug fixes
- Pros:** Rigorous, suitable for large projects with strict release cycles.
- Cons:** Can be complex for smaller projects.

4. Forking Workflow

- Popular:** Used in open-source projects.
- Process:** Developers fork the main repository, make changes, and submit a pull request to merge their changes back into the main repository.
- Pros:** Encourages community contributions, decentralized development.
- Cons:** Can be challenging to manage for large projects.

5. OneFlow Workflow

- Simplified:** A single branch (usually main) is used for all development and releases.
- Pros:** Minimal overhead, suitable for small projects.
- Cons:** Can be risky for large projects, as there's no clear separation between development and production.

Branching benefits

1.Isolation of Changes:

1. **Experimentation:** Create branches to experiment with new features or ideas without affecting the main codebase.
2. **Risk Mitigation:** If a change goes wrong, you can easily revert to a previous state.

2.Parallel Development:

1. **Multiple Teams:** Different teams can work on separate branches simultaneously.
2. **Feature Development:** Multiple features can be developed independently.

3.Code Review and Collaboration:

1. **Pull Requests:** Create pull requests to share changes and request feedback from others.
2. **Collaboration:** Collaborate effectively on code changes.

4.Easier Rollbacks:

1. **Quick Reversions:** If a change causes issues, you can easily revert to a previous branch.

5.Release Management:

1. **Staging Areas:** Use branches as staging areas for releases.
2. **Hotfixes:** Apply urgent fixes to a specific branch without affecting ongoing development.

6.Experimentation and Innovation:

1. **Risk-Free Exploration:** Try out new ideas without fear of breaking the main codebase.

7.Feature Flags:

1. **Conditional Features:** Enable or disable features based on flags.

8.Integration and Testing:

1. **Integration Branches:** Combine changes from different branches for testing.

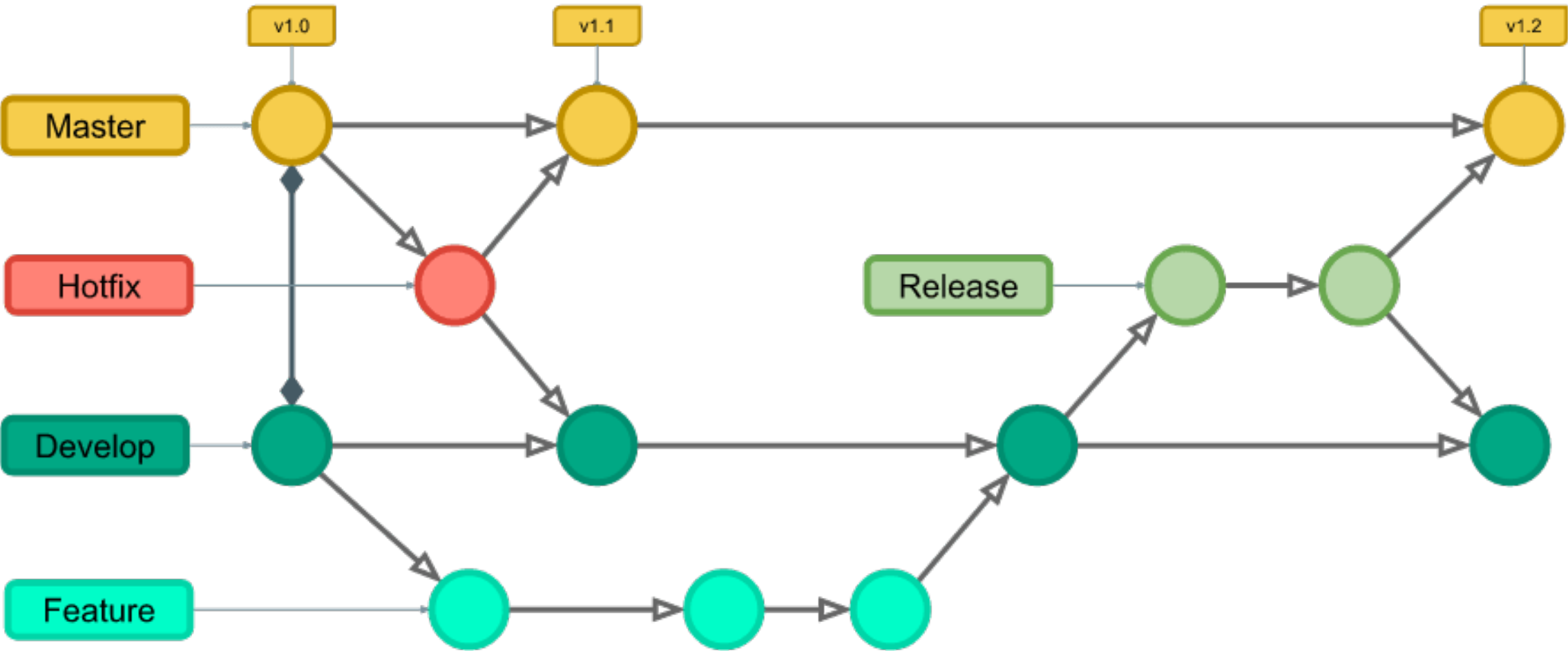
9.Code Quality:

1. **Refactoring:** Refactor code on a separate branch without affecting the main codebase.

10.Version Control:

- Multiple Versions:** Maintain multiple versions of your codebase.

Git Branching - GitFlow



Git Commands

Command	What is the purpose of each?
<code>git config --global user.name "<name>"</code>	Set a global username for Git in your local system
<code>git config --global user.email "<emailID>"</code>	Set a global email for Git in your local system
<code>git init</code>	Initialize a new git repo
<code>git clone <repository_url></code>	Create a working copy of a remote git repo
<code>git add <file_name></code>	Move the changes to the staging area
<code>git commit -m "<message>"</code>	Commit the changes to the repo
<code>git push <remote> <branch_name></code>	Push local changes to the remote repo
<code>git branch <branch_name></code>	Create a new branch in the repo
<code>git checkout <branch_name></code>	Switch to a different branch
<code>git checkout -b <branch_name></code>	Create and switch to a new branch

Git Commands

git branch	List all branches of the repo
git branch -d/-D <branch_name>	Delete a branch from the repo -d: delete a branch that has been fully merged into the current branch. -D: forcefully delete a branch, even if it contains unmerged changes.
git log	View the commit history
git merge <branch_name>	Merge changes from one branch to another
git pull <remote> <branch_name>	Fetch and merge changes from the remote repo
git fetch	Fetch changes from the remote repo
git status	Check the state of the repo and the files within
git diff	Show differences between the current work and the last saved snapshot in the repo

Git Commands

git reset	Undo changes in the repo
git revert <commitID>	Undo a specific commit by creating a new commit
git remote add <name> <repository_url>	Connect the local repo to the remote repo
git remote -v	Displays the list of remote repos associated with local repo, along with their corresponding URLs
git stash	Temporarily save changes without committing
git stash pop	Apply & remove the most recent stash changes
git stash apply	Apply the most recent stash changes
git cherry-pick <commitID>	Apply specific commits from one branch onto another
git clean	Remove untracked files and directories from the local working directory

1. git init

This command let us **create a new repository**.
A hidden **.git** directory is added to the folder.

Most of the git command do not work outside
initialized project , so this is the first
command you will run in a project

Go to project folder > run git init

A terminal window with a dark background and a light blue border. At the top, there are three colored circles: red, yellow, and green. Below them, the command '\$ git init' is written in a light blue monospace font.

```
$ git init
```

2. git clone

This command **creates a local copy** of a **remote repository** .

When you clone a repo the source code gets automatically downloaded to local machine. This local repo will point to remote repo and can **PUSH** and **PULL** changes to it



```
$ git clone < git-repo-url>
```

3. git add

This command **add your changes** to **staging area** where you can compare you local version with remote repo code.

It is **mandatory to stage the code** before commit(push to remote) using git add command.

To stage all files use (.) - **git add .** in the same repo



```
$ git add <file-1> <file-2>
```

4. git commit

This command **saves your changes** to your local repository.

Everytime you commit you have to add a **small message** about the changes you made. This will help to keep track of the changes later.



```
$ git commit -m "commit-message"
```

5. git push

This command **push your changes** from local repository to your remote repository. One can only push the committed changes.

It also **creates the repository** with the branch name you enter if repository does not exist on remote location.

If branch is already connected to remote then run - **git push**



```
$ git push <remote> <branch-name>
```

6. git pull

This command **fetches latest changes** from remote repository to your local. This is helpful when multiple people are working on same repository. It will help to keep your local repo **updated with latest code.**

If branch is already connected to remote then run - **git pull**



```
$ git pull <remote> <branch-name>
```

7. git checkout

This command helps to **switch to an existing branch or create a new branch**.

Before checking out make sure the **branch exist in your local machine** and the changes in the current branch is already **staged or committed**.



```
$ git checkout -b <branch>
```

.gitignore file

The .gitignore file tells Git which files to ignore when committing your project to the GitHub repository.

The .gitignore is located in the root directory of your repo.

Reference =

<https://github.com/github/gitignore>

<https://git-scm.com/docs/gitignore>

Git Demo

Install Git Client

Install Git

yum install git

```
[root@ip-172-31-9-99 ~]# yum install git
Last metadata expiration check: 0:00:21 ago on Tue Aug 22 09:54:39 2023.
Dependencies resolved.
=====
Package                                Architecture      Version            Repository          Size
=====
Installing:
git                                    x86_64            2.40.1-1.amzn2023.0.1  amazonlinux         57 k
Installing dependencies:
git-core                             x86_64            2.40.1-1.amzn2023.0.1  amazonlinux         4.3 M
git-core-doc                         noarch            2.40.1-1.amzn2023.0.1  amazonlinux         2.6 M
perl-Error                           noarch            1:0.17029-5.amzn2023.0.2 amazonlinux          41 k
perl-File-Find                       noarch            1.37-477.amzn2023.0.5  amazonlinux          26 k
perl-Git                             noarch            2.40.1-1.amzn2023.0.1  amazonlinux          45 k
perl-TermReadKey                     x86_64            2.38-9.amzn2023.0.2    amazonlinux          36 k
perl-lib                             x86_64            0.65-477.amzn2023.0.5  amazonlinux          15 k
=====
Transaction Summary
=====
Install 8 Packages

Total download size: 7.1 M
Installed size: 34 M
```

git --version

```
[root@ip-172-31-9-99 gitdemo]# git --version
git version 2.40.1
[root@ip-172-31-9-99 gitdemo]#
```

Git config

```
git config --global user.name "Saurabh Porwal"
```

```
git config --global user.email "devopscloudlabs@gmail.com"
```

```
[root@ip-172-31-9-99 gitdemo]# git config --global user.name "Saurabh Porwal"  
[root@ip-172-31-9-99 gitdemo]# git config --global user.email "devopscloudlabs@gmail.com"  
[root@ip-172-31-9-99 gitdemo]#
```

Edit Configuration here, if needed using this command :

```
git config --global -e
```

More Config, Reference <https://git-scm.com/docs/git-config>

Add ssh key

```
ssh-keygen -t ed25519 -C "yourname@email.com"
```

```
[ec2-user@ip-172-31-9-99 ~]$ ssh-keygen -t ed25519 -C
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/ec2-user/.ssh/id_ed25519):
/home/ec2-user/.ssh/id_ed25519 already exists.
Overwrite (y/n)?
[ec2-user@ip-172-31-9-99 ~]$ ssh-keygen -t ed25519 -C
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/ec2-user/.ssh/id_ed25519):
/home/ec2-user/.ssh/id_ed25519 already exists.
Overwrite (y/n)? y
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/ec2-user/.ssh/id_ed25519.
Your public key has been saved in /home/ec2-user/.ssh/id_ed25519.pub.
The key fingerprint is:
SHA256:AEYh4C4iPLlvldTFD8aprsff/M+1Dk8aI3Sy+Fae56M sau
The key's randomart image is:
+--[ED25519 256]--+
|.....=..o..|
|.o..B|
|.o.+o|
|o..+|
|+=..oso|
|+o..o+|
|.o..oo+..|
|..o..+oB+..|
|..o..oE**+|
+-----[SHA256]-----+
```

```
Get the ssh key
```

```
[ec2-user@ip-172-31-9-99 ~]$ cat /home/ec2-user/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIAYGgOWK1hpIcTsNL0Lvo1AO8anngkAZlPgoBze7Pymp
```

Add ssh key in GitHub profile

Go to your GitHub.com profile settings(top right menu) and add the rsa.pub in “New SSH Key”

github.com/settings/ssh/new

Settings

Q Type [?] to search

Go to your personal profile

Public profile

Account

Appearance

Accessibility

Notifications

Access

Billing and plans

Emails

Password and authentication

Sessions

SSH and GPG keys

Organizations

Enterprises

Moderation

Code, planning, and automation

Repositories

Title

ssh-sp

Key type

Authentication Key

Key

sh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIAYGgQWKlhplcTsNL0Lvo1A08annngkAZIPgoBze7Pymp
yourname@example.com

Add SSH key

github.com/settings/keys

Settings

Q Type [?] to search

Go to your personal profile

Public profile

Account

Appearance

Accessibility

Notifications

Access

Billing and plans

Emails

Password and authentication

Sessions

SSH and GPG keys

Organizations

Enterprises

Moderation

Code, planning, and automation

Repositories

Codespaces

Packages

Copilot

Pages

Saved replies

SSH keys

New SSH key

This is a list of SSH keys associated with your account. Remove any keys that you do not recognize.

Authentication Keys

ssh-sp

SHA256: AEYh4C4iPLvLdTFD8aprsff/M+Ldk8aI3Sy+Fae56M

Added on Aug 22, 2023

Last used within the last week — Read/write

Delete

Check out our guide to [generating SSH keys](#) or troubleshoot [common SSH problems](#).

GPG keys

New GPG key

There are no GPG keys associated with your account.

Learn how to [generate a GPG key](#) and [add it to your account](#).

Vigilant mode

☐ Flag unsigned commits as unverified

This will include any commit attributed to your account but not signed with your GPG or S/MIME key.
Note that this will include your existing unsigned commits.

[Learn about vigilant mode.](#)

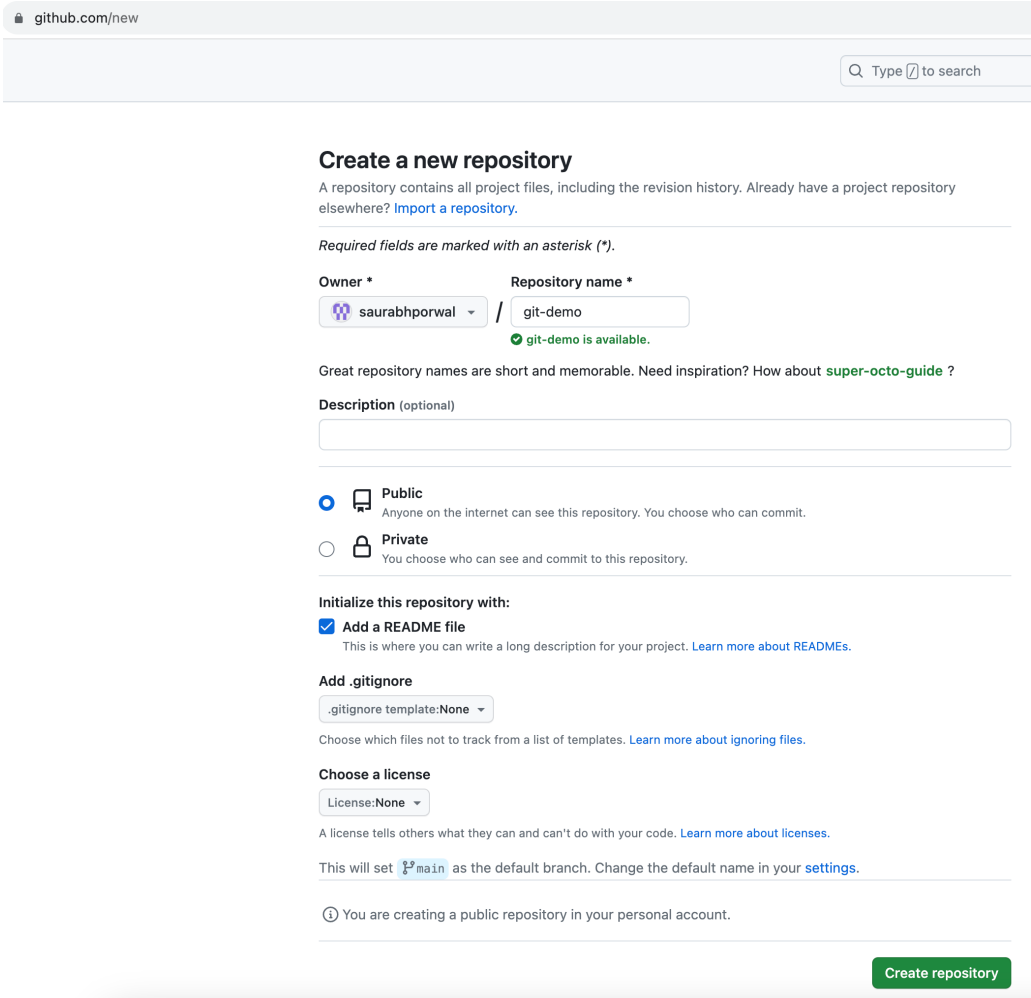
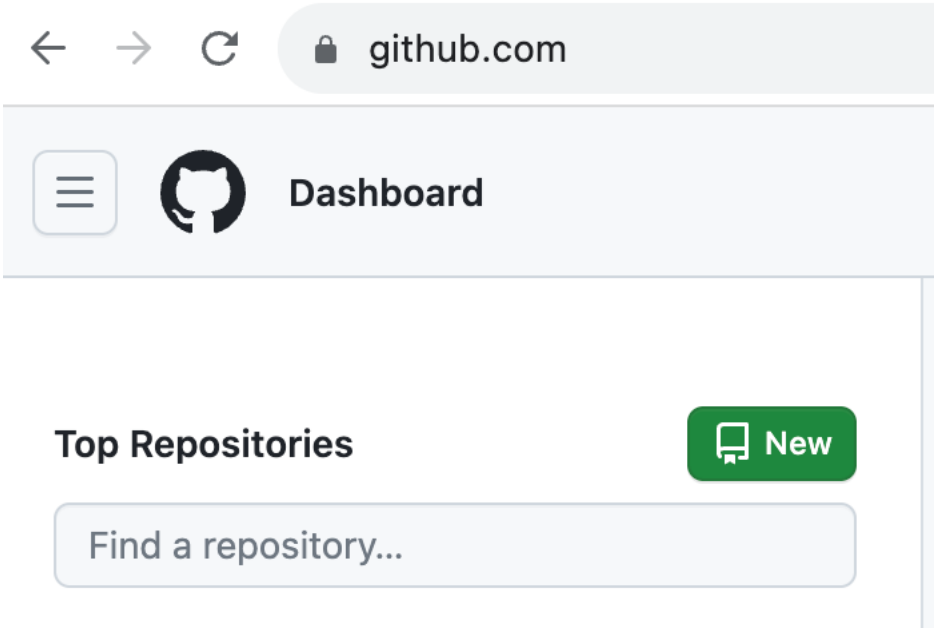
39

https://topmate.io/cloud

https://www.youtube.com/@DevOps-Cloud

Create new repo in GitHub

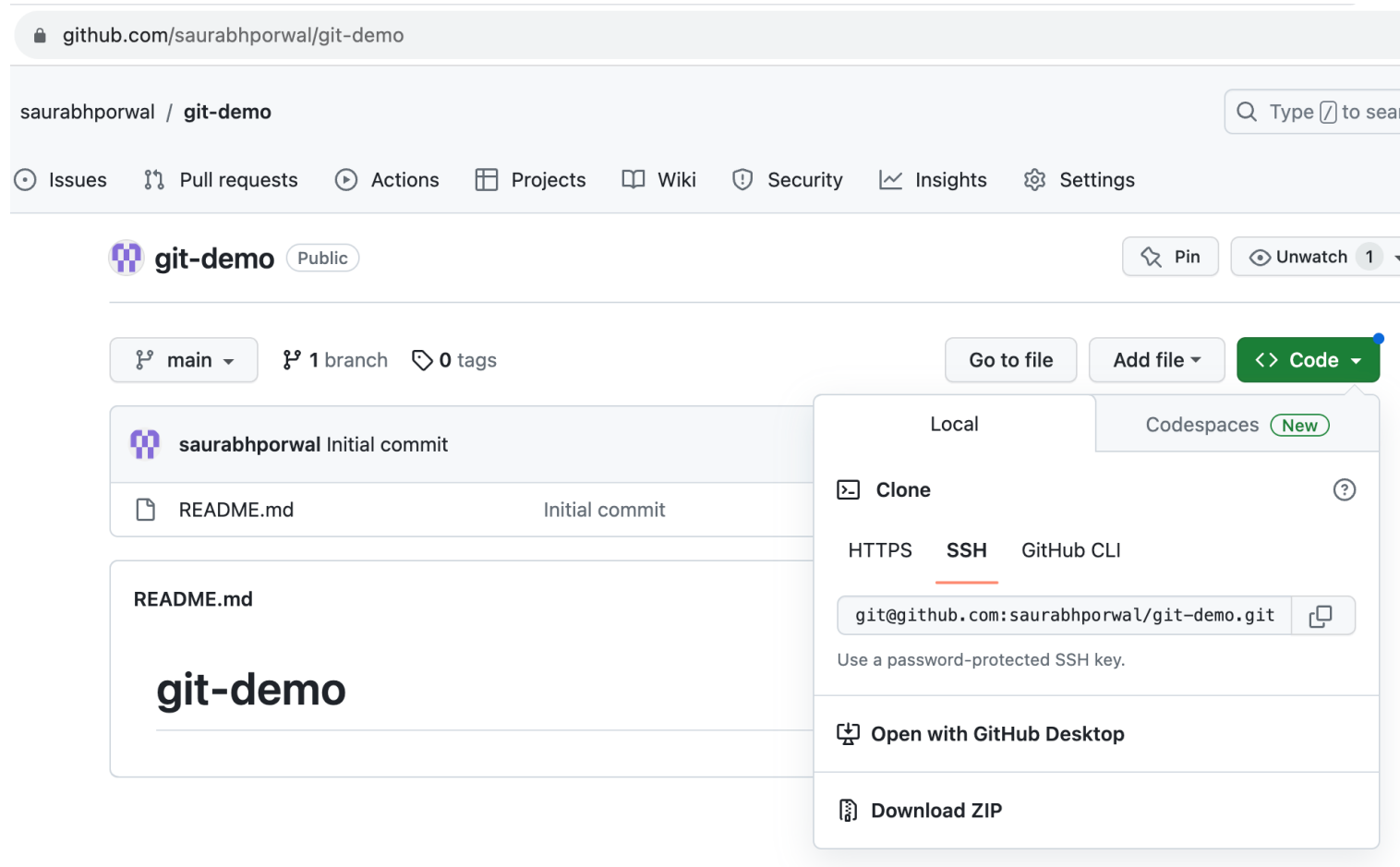
GitHub dashboard, create “New” repository



Get the ssh URL from GitHub

Get the ssh url

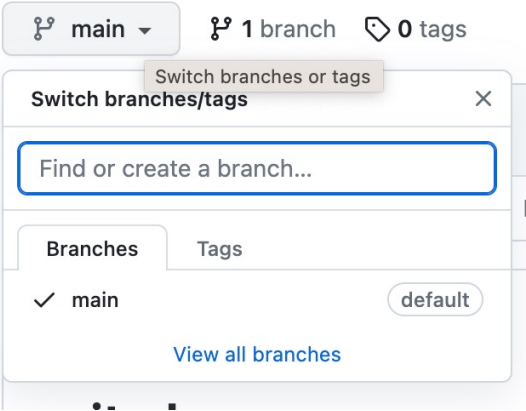
git@github.com:saurabhporwal/git-demo.git



Clone repo in local machine

```
git clone git@github.com:saurabhporwal/git-demo.git
```

```
[ec2-user@ip-172-31-9-99 ~]$ git clone git@github.com:saurabhporwal/git-demo.git
Cloning into 'git-demo'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
Receiving objects: 100% (3/3), done.
[ec2-user@ip-172-31-9-99 ~]$
```



git branch

```
[ec2-user@ip-172-31-9-99 ~]$ ls
git-demo
[ec2-user@ip-172-31-9-99 ~]$ cd git-demo/
[ec2-user@ip-172-31-9-99 git-demo]$ git branch
* main
[ec2-user@ip-172-31-9-99 git-demo]$ ls -la
total 4
drwxr-xr-x. 3 ec2-user ec2-user 35 Aug 22 12:33 .
drwx-----. 4 ec2-user ec2-user 143 Aug 22 12:35 ..
drwxr-xr-x. 8 ec2-user ec2-user 163 Aug 22 12:33 .git
-rw-r--r--. 1 ec2-user ec2-user 10 Aug 22 12:33 README.md
[ec2-user@ip-172-31-9-99 git-demo]$
```

Create new branch and Add Files to Repository

```
git checkout -b "p1feature"
```

```
git add .
```

```
git commit -m "Adding  
sample files"
```

```
[ec2-user@ip-172-31-9-99 git-demo]$ git checkout -b "p1feature"
Switched to a new branch 'p1feature'
[ec2-user@ip-172-31-9-99 git-demo]$ echo "This is first file" > file1.txt
[ec2-user@ip-172-31-9-99 git-demo]$ echo "This is Second file" > file2.txt
[ec2-user@ip-172-31-9-99 git-demo]$ git add .
[ec2-user@ip-172-31-9-99 git-demo]$ git commit -m "Adding sample files"
[p1feature e9b58f0] Adding sample files
Committer: EC2 Default User <ec2-user@ip-172-31-9-99.ap-southeast-2.compute.internal>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

2 files changed, 2 insertions(+)
create mode 100644 file1.txt
create mode 100644 file2.txt
[ec2-user@ip-172-31-9-99 git-demo]$
```

Push new branch to remote origin

```
git push --set-upstream origin  
p1feature
```

```
[ec2-user@ip-172-31-9-99 git-demo]$ git push --set-upstream origin p1feature  
Enumerating objects: 5, done.  
Counting objects: 100% (5/5), done.  
Compressing objects: 100% (2/2), done.  
Writing objects: 100% (4/4), 384 bytes | 384.00 KiB/s, done.  
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0  
remote:  
remote: Create a pull request for 'p1feature' on GitHub by visiting:  
remote:      https://github.com/saurabhpurwal/git-demo/pull/new/p1feature  
remote:  
To github.com:saurabhpurwal/git-demo.git  
 * [new branch]      p1feature -> p1feature  
branch 'p1feature' set up to track 'origin/p1feature'.  
[ec2-user@ip-172-31-9-99 git-demo]$
```

New branch is created in remote

New branch
“p1feature” is created
in GitHub

github.com/saurabhporwal/git-demo

saurabhporwal / git-demo

Type ↵ to search

Issues

Pull requests

Actions

Projects

Wiki

Security

Insights

Settings

git-demo

Public

Pin

Unwatch 1

p1feature had recent pushes 1 minute ago

Compare & pull request

main ▾

2 branches

0 tags

Go to file

Add file ▾

Code ▾

Switch branches/tags

Find or create a branch...

Branches

Tags

main default

p1feature

View all branches

Initial commit

742ee22 10 minutes ago 1 commit

README.md

git-demo

Compare and Pull request

Branch "p1feature" contains the files added through linux

Click on "Compare & pull request"

github.com/saurabhporwal/git-demo/tree/p1feature

saurabhporwal / git-demo

Type ↵ to search

Issues

Pull requests

Actions

Projects

Wiki

Security

Insights

Settings

 **git-demo** Public

Pin

Unwatch 1

 **p1feature** had recent pushes 2 minutes ago

Compare & pull request

p1feature

2 branches

0 tags

Go to file

Add file

<>

Code

This branch is 1 commit ahead of main.

Contribute

EC2 Default User Adding sample files

e9b58f0 4 minutes ago 2 commits

README.md

Initial commit

11 minutes ago

file1.txt

Adding sample files

4 minutes ago

file2.txt

Adding sample files

4 minutes ago

README.md

git-demo

46

https://topmate.io/cloud

https://www.youtube.com/@DevOps-Cloud

Open a Pull request

This will merge the files from “p1feature” branch to “main”.

github.com/saurabhporwal/git-demo/compare/p1feature?expand=1

saurabhporwal / git-demo

Issues Pull requests Actions Projects Wiki Security Insights Settings

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#).

base: main ← compare: p1feature

✓ Able to merge. These branches can be automatically merged.

Adding sample files

Write Preview

H B I

This will merge the files from p1feature branch to main.

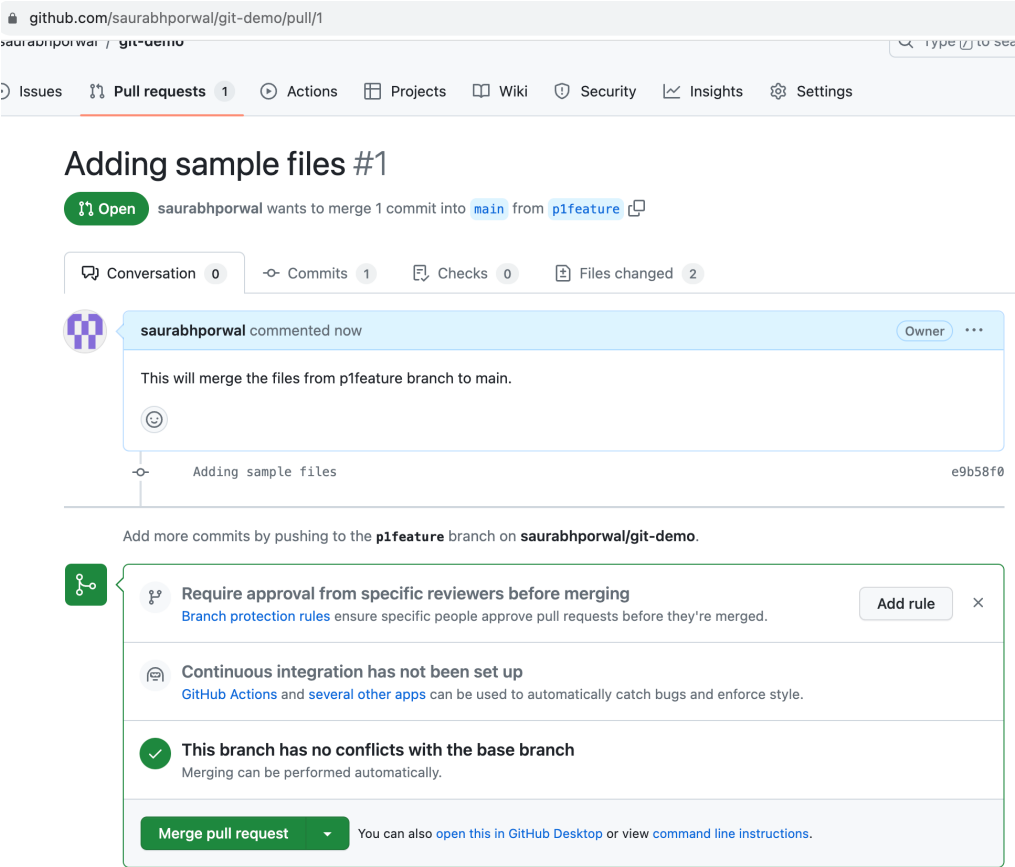
Attach files by dragging & dropping, selecting or pasting them.

Create pull request

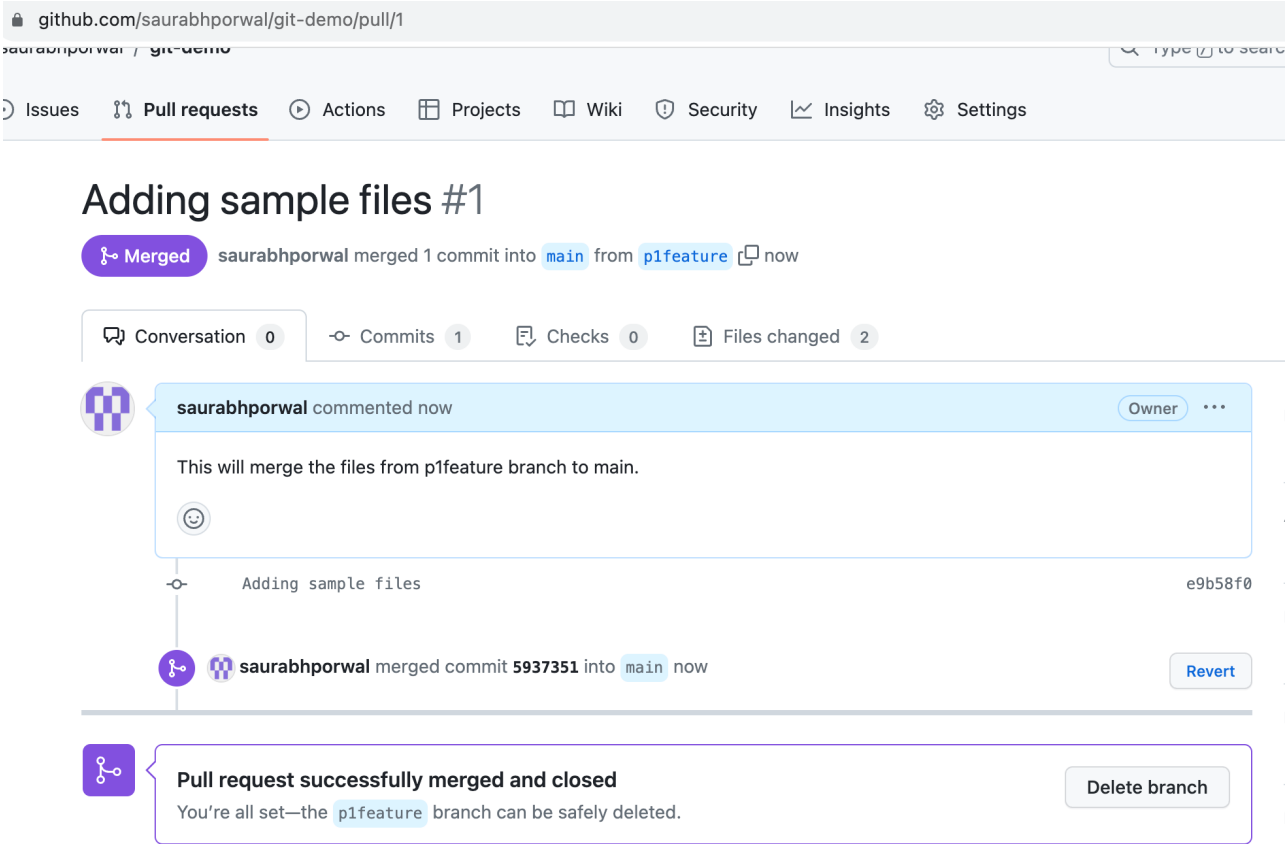
Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

Merge Pull Request

Merge Pull Request



Pull request successfully merged



Main branch Pull request Complete

Main branch now contains all the changed files from p1feature branch

github.com/saurabhporwal/git-demo

saurabhporwal / git-demo

Issues Pull requests Actions Projects Wiki Security Insights Settings

git-demo Public Pin Unwatch 1

main 2 branches 0 tags Go to file Add file Code

 Your main branch isn't protected

Protect this branch from force pushing or deletion, or require status checks before merging. [Learn more](#)

Protect this branch

 saurabhporwal Merge pull request #1 from saurabhporwal/p1feature

5937351 now 3 commits

README.md	Initial commit	17 minutes ago
file1.txt	Adding sample files	10 minutes ago
file2.txt	Adding sample files	10 minutes ago

README.md

git-demo

Git Commands

<code>git init</code>	Initialize a local Git repository
<code>git clone</code> <code>ssh://git@github.com/[username]/[repository-name].git</code>	Create a local copy of a remote repository
<code>git status</code>	Check status
<code>git add [file-name.txt]</code>	Add a file to the staging area
<code>git add -A</code>	Add all new and changed files to the staging area
<code>git commit -m "[commit message]"</code>	Commit changes
<code>git rm -r [file-name.txt]</code>	Remove a file (or folder)
<code>git branch</code>	List branches (the asterisk denotes the current branch)
<code>git branch -a</code>	List all branches (local and remote)
<code>git branch [branch name]</code>	Create a new branch

<code>git branch -d [branch name]</code>	Delete a branch
<code>git push origin --delete [branch name]</code>	Delete a remote branch
<code>git checkout -b [branch name]</code>	Create a new branch and switch to it
<code>git checkout [branch name]</code>	Switch to a branch
<code>git checkout -</code>	Switch to the branch last checked out
<code>git push</code>	Push changes to remote repository (remembered branch)
<code>git push origin --delete [branch name]</code>	Delete a remote branch
<code>git pull</code>	Update local repository to the newest commit
<code>git pull origin [branch name]</code>	Pull changes from remote repository
<code>git remote add origin</code> <code>ssh://git@github.com/[username]/[repository-name].git</code>	Add a remote repository
<code>git remote set-url origin</code> <code>ssh://git@github.com/[username]/[repository-name].git</code>	Set a repository's origin branch to SSH

Reference: <https://github.com/joshnh/Git-Commands>

Industry Best Practices

1. Use a Version Control System: Always use Git or another version control system to track changes to your code. Never work directly on the master (or main) branch.

2. Branching Strategy:

1. **Master/Main Branch:** Keep the master (or main) branch stable and deployable at all times.
2. **Feature Branches:** Create a new branch for each feature or bug fix. Use descriptive branch names (e.g., feature/user-authentication).
3. **Pull Requests/Merge Requests:** Use pull requests (or merge requests) to propose and review changes before merging them into the main branch.

3. Commit Frequently:

1. Make small, focused commits with descriptive commit messages.
2. Avoid committing large, unrelated changes together.

4. Commit Messages:

1. Follow a consistent format for commit messages, such as the "Conventional Commits" format or a custom format that your team agrees upon.
2. Start the message with a one-line summary followed by a more detailed description if necessary.

5. Code Review:

1. Conduct code reviews for all changes before merging.
2. Use code review tools like GitHub/GitLab's built-in features or third-party tools like Reviewable or Crucible.

6. Gitignore:

1. Use a .gitignore file to specify which files and directories should be excluded from version control (e.g., build artifacts, IDE-specific files).

Industry Best Practices

7. Rebase vs. Merge:

1. Use git rebase for feature branches to keep a linear and clean commit history.
2. Use git merge for incorporating changes from the main branch into your feature branch.

8.Pull/Push Regularly:

1. Pull the latest changes from the main branch regularly to stay up to date.
2. Push your changes to the remote repository frequently to avoid merge conflicts.

9.Use Tags:

1. Create tags to mark important releases or versions of your software.

10.Keep Commit History Clean:

1. Avoid amending or force-pushing commits to the main branch once they are shared with others.
2. Use git revert or create a follow-up commit to fix mistakes.

11.Documentation:

1. Include documentation in your repository (e.g., README.md) to explain how to set up and use the project.
2. Document any branching strategies or conventions your team follows.

12.Continuous Integration (CI):

1. Set up a CI pipeline to automatically run tests and checks on all pull requests.
2. Ensure that the CI pipeline fails if there are linting errors or failing tests.

Industry Best Practices

13. Git Hooks:

1. Consider using Git hooks to automate checks (e.g., linting, code formatting) before commits or pushes.

14.Security:

1. Avoid storing sensitive information (e.g., API keys, passwords) in code repositories.
2. Use tools like Git-Secret or Git-Crypt for managing secrets.

15.Backup and Recovery:

1. Regularly back up your Git repositories, including remote copies.
2. Plan for disaster recovery, including scenarios like accidental data loss.

16.Training and Onboarding:

1. Ensure that team members are familiar with Git and your team's specific workflows through training and onboarding processes.

17.Versioning Conventions:

1. Use semantic versioning (e.g., MAJOR.MINOR.PATCH) to tag releases.

Remember that these best practices can be adapted to fit your team's specific needs and workflows. Consistency and communication within your team are key to successful Git usage.



Assignments

#	Base Goals	Extended Goals
1	Try out all Base Linux commands	• Run all advanced Linux commands • Write a Shell Script and execute a for/while loop in it • Change ownership and execute permission of files
2	Install Git Client and configure the repository	• -
3	Create new branch and add files and commit	• -
4	Push files to remote repository	• -
5	Create a pull request and merge to Master	
6	• -	• Create a private Git repository and use Groups to assign permissions. • Set up a Git reviewer approval process in pull requests • Create multiple branches, change same file and merge to master. Resolve merge conflicts.
7	• -	Run Advanced Git commands as defined in https://earthly.dev/blog/advanced-git-commands/

Project1 - LinuxWebGit: Building and Managing a Basic Website with Linux and Git(1)

Step 1: Prepare Your Linux Environment

- 1.Select a Linux Distribution:** Choose a Linux distribution like Ubuntu, CentOS, or Debian.
- 2.Install Linux:** Install your chosen Linux distribution on a virtual machine using software like VirtualBox or VMware. Alternatively, consider a cloud-based solution such as AWS EC2 or Google Cloud Platform.
- 3.Get Comfortable with Linux Basics:** Familiarize yourself with fundamental Linux commands such as ls, cd, mkdir, cp, mv, rm, sudo, and package managers like apt-get (or yum). Numerous online resources and tutorials can help you grasp these essentials.

Step 2: Create a Simple Website

- 1.Install a Web Server:** Set up a web server like Apache or Nginx on your Linux machine using package managers like apt (for Ubuntu/Debian) or yum (for CentOS/RHEL).
- 2.Craft a Basic HTML Page:** Develop a straightforward HTML page for your website using any text editor such as nano or vim.
- 3.Place HTML File in Web Server Directory:** Put your HTML file in the directory from which your web server serves files. For Apache, this might be /var/www/html/, and for Nginx, it could be /usr/share/nginx/html/.
- 4.Preview Your Website:** Access your website through a web browser by entering the IP address or domain name of your Linux server.

Step 3: Set Up Git

- 1.Install Git:** Install Git on your Linux server using your package manager (apt, yum, etc.).
- 2.Configure Git:** Personalize Git with your name and email address using the following commands:

```
git config --global user.name "Your Name"
git config --global user.email "youremail@example.com"
```

Step 4: Version Control Your Website with Git

Initialize a Git Repository: Navigate to your website's directory and initialize a Git repository:

```
cd /path/to/your/website
git init
```

Add Files to the Repository: Add your website files to the Git repository:

```
git add .
```

Commit Your Changes: Commit your changes with a descriptive message:

```
git commit -m "Initial commit: Added basic HTML page"
```

Project1 - Building and Managing a Basic Website with Linux and Git(2)

Step 5: Explore Git Workflow

Create Branches: Experiment with new features or changes by creating a new branch:

```
git checkout -b feature-branch
```

Make Changes: Modify your website files.

Commit Changes: Add and commit your changes to the feature branch.

Merge Branches: Merge the feature branch back into the main branch (master).

Step 6: Collaborate Using Git

Create a Remote Repository: Establish a repository on a Git hosting service like GitHub, GitLab, or Bitbucket.

Add Remote: Incorporate the remote repository URL into your local Git repository:

```
git remote add origin <remote-repository-url>
```

Push Changes: Push your local repository to the remote repository:

```
git push -u origin master
```

Collaborate with Others: Invite collaborators to join your project by granting them access to the remote repository.

Pull Changes: Synchronize with the latest changes made by collaborators by pulling updates from the remote repository:

```
git pull origin master
```

The output of the project would be a functional basic website hosted on a Linux server, managed and version-controlled using Git.

To validate the success of the project, you can perform the following steps:

Access the Website: Open a web browser and navigate to the IP address or domain name of your Linux server to ensure that the website is accessible.

By following these steps, you'll gain practical experience with Linux commands, setting up a web server, managing a simple website, version control using Git, collaboration workflows, and potentially continuous integration and deployment practices.

Project2 : Automated Backup and Archiving System with Shell Script

Project Description: This project involves developing a standalone complex shell script to automate the backup and archiving process for critical files and directories on a Linux system. The script will include functionalities such as creating backups, compressing files, encrypting sensitive data, managing retention policies, and sending notifications.

Project Tasks:

1.Setup Development Environment:

1. Choose a Linux distribution for development and testing.
2. Identify the directories and files to be included in the backup. You may choose the files from Project1.

2.Design Script Structure:

1. Plan the structure of the shell script, outlining functions and workflows.
2. Determine command-line options for user interaction (e.g., backup frequency, retention period).

3.Write Backup and Archiving Logic:

1. Develop functions to:
 1. Create backup directories with timestamped names.
 2. Copy selected files and directories to the backup location.
 3. Compress backup files into a single archive.
 4. Encrypt sensitive data within the archive using GPG.
 5. Apply retention policies to manage backup storage space.

4.Implement Error Handling and Logging:

1. Incorporate error handling mechanisms to detect and handle issues during backup and archiving.
2. Implement logging functionality to record script activities, errors, and notifications.

5.Configure Notification System:

1. Set up email notifications to alert administrators about backup status, errors, and warnings.
2. Include options to customize notification recipients and message content.

6.Test Script Functionality:

1. Test the shell script with sample data to ensure that backups are created and archived correctly.
2. Validate error handling and logging mechanisms.

7.Security Considerations:

1. Ensure secure handling of sensitive data, such as encryption keys and backup archives.
2. Implement access controls to restrict script execution and data access.

8.Documentation and Usage Guide:

1. Create comprehensive documentation explaining script usage, command-line options, and configuration settings.
2. Include examples and best practices for configuring the backup system.
3. Provide troubleshooting tips and common error resolutions.

Expected Output:

- A standalone complex shell script capable of automating the backup and archiving process for critical files and directories.
- Integration of functionalities such as backup creation, compression, encryption, retention policy management, error handling, logging, and notification.
- Secure handling of sensitive data and implementation of access controls.
- Comprehensive documentation for users, including installation instructions, usage guide, configuration settings, and troubleshooting tips.

Validation:

- Successful execution of the shell script to create backups and archives.
- Verification of backup integrity and encryption.
- Notification alerts for backup status and errors.
- Documentation completeness and clarity, ensuring users can understand and utilize the script effectively.

Project3 - “Collaborative Shell” Git and Linux Project

Project Description:

Overview:

The Git Skills Test - Collaborative Text File project is a hands-on exercise designed to evaluate your proficiency with Git, the version control system widely used for collaborative software development. In this project, you will work with a simple text file, demonstrating your ability to create branches, make changes, resolve conflicts, and collaborate with others in a Git-based environment.

Project Goals:

- 1. Git Fundamentals:** Gain practical experience with Git by performing basic version control operations such as cloning, branching, committing, and merging.
- 2. Collaboration:** Simulate a collaborative workflow by making changes to a shared **shell script** file and managing potential conflicts when multiple contributors make changes concurrently.
- 3. Documentation:** Practice good Git habits by creating informative commit messages and keeping a concise project README.

Features:

The “Collaborative Shell” Text File project will include the following features:

- 1. Repository Setup:** A Git repository will be created, containing a shared shell script file.
- 2. Collaborative Editing:** You and other contributors (real or simulated) will make changes to the file on separate branches.
- 3. Branch Merging:** You will merge changes from multiple branches, resolving any conflicts that arise.
- 4. Commit Messages:** Detailed commit messages will be required for each change made.

How to Use:

- 1. Clone the Repository:** Use git clone to create a local copy of the Git repository for this project.
- 2. Create Your Branch:** Create a new branch for yourself using git checkout -b your-branch-name.
- 3. Make Changes:** Edit the file in your branch and commit your changes with meaningful commit messages.
- 4. Simulate Collaboration:** Invite other contributors (or use multiple terminal instances) to make changes on their branches simultaneously.
- 5. Branch Merging:** Merge changes from different branches into the main branch, resolving any conflicts as necessary.
- 6. Document Your Work:** Update the README file in the repository with information about your contributions.

Skills to Test:

- Cloning a Git repository.
- Creating and switching between branches.
- Testing out creating a Linux shell script
- Making changes, committing, and pushing to remote branches.
- Resolving merge conflicts.
- Creating informative commit messages.
- Collaborative Git workflow.

Extensions:

To challenge yourself further, consider adding the following advanced features:

- Implement a review process where changes must be approved before merging.

Conclusion:

The Git Skills Test - Collaborative Text File project offers a practical opportunity to showcase your Git skills and understanding of collaborative version control. By successfully completing this project, you'll demonstrate your ability to work effectively with Git in a team environment and manage version control for collaborative coding projects.